

AI audit report

I use AI tools for the following tasks.

This report contains only my interactions with the three helper AIs used to generate test drafts. It does not include any conversation between the homework owner and the repository assistant.

Interaction summary

Interaction	AI tool	Date and time	Task
1	Codex helper AI (Hegel)	2026-08-18 12:25:47 +07	Generate 35 FR-03 cases
2	Codex helper AI (Banach)	2026-08-18 12:25:59 +07	Generate 35 FR-09 cases
3	Codex helper AI (Raman)	2026-08-18 12:26:09 +07	Generate 35 FR-12 cases

Other tools

- Postman organized the requests, variables, pre-request scripts, and test scripts.
- Newman 6.2.2 ran the collection and produced the HTML and JSON reports.
- GitHub Actions ran the CI/CD pipeline.

Human review

I checked all 105 generated cases. I marked 89 as valid, 3 as invalid, and 13 as incomplete. I corrected the invalid or incomplete test oracles and added 15 human cases. The final decisions, execution results, and bug reports are my responsibility.

Full AI interaction logs

The three logs below contain each helper's name, date and time, my full prompt, and the full AI output.

AI Interaction Log — FR-03

- Tool name: Codex helper AI (Hegel)
- Date and time: 2026-08-18 12:25:47 +07 (Asia/Ho_Chi_Minh)

Prompt

You are a helper AI being guided by a third-year software engineering student. Inspect eshop-sut/api_specification.md and the relevant implementation only to understand FR-03 (POST /api/forgot-password and POST /api/reset-password). Do not edit any files. Produce exactly 35 AI-generated test-case drafts in a compact Markdown table with columns: ID, endpoint, category, precondition, request/input, expected status, expected schema/behavior. Cover domain partitions for every parameter, the forgot↔reset state flow, token reuse/invalid transitions, SEC-01–SEC-07 where relevant, and exact response schema. Base expected behavior on the API specification, clearly flag any assumption, and keep the output concise enough to paste into an AI audit log.

AI output

Assumptions: A1 — because the API specification omits validation/error contracts, invalid input is expected to return 400 {"error": "<string>"}; unknown forgot-password email follows the implementation contract 404 {"error": "User not found"}. A2 — the reset success schema is omitted, so the observed implementation schema is used. A3 — SEC-07 requires expiry but gives no lifetime; “expired” means older than the configured lifetime. A4 — email matching is assumed case-insensitive. SEC-02/03/06 are not applicable because password recovery must work without JWT and is neither an admin nor profile-update API. confirmPassword is a UI field, not an API parameter.

ID	endpoint	category	precondition	request/input	expected status	expected schema/behavior
AI-FR03-01	forgot	valid/schema	Registered test@eshop.com	{"email": "test@eshop.com"}	200	Exact object: message="Mã reset lỗi khi user đã đặt token" and string resetToken matching ^\d{6}\$; no extra fields.
AI-FR03-02	forgot	email existence	Email is not registered	{"email": "nobody@example.com"}	404	Exact {"error": "User not found"}; no token created.
AI-FR03-03	forgot	email required	None	{}	400	[A1] Exact error-only object; no account changes.
AI-FR03-04	forgot	email empty	None	{"email": ""}	400	[A1] Reject empty string; exact error-only object.
AI-FR03-05	forgot	email whitespace	Registered email exists	{"email": " "}	400	[A1] Reject whitespace-only value; no token created.
AI-FR03-06	forgot	email format	None	{"email": "test-at-eshop"}	400	[A1] Reject malformed email; exact error-only object.
AI-FR03-07	forgot	email type	None	{"email": null}	400	[A1] Reject null; no server error.
AI-FR03-08	forgot	email type	None	{"email": 12345}	400	[A1] Reject non-string number.
AI-FR03-09	forgot	email type	None	{"email": {"\$ne": null}}	400	[A1] Reject object input; no query-operator interpretation.

ID	endpoint	category	precondition	request/input	expected status	expected schema/behavior
AI-FR03-10	forgot	email boundary	None	Email string longer than 254 characters	400	[A1] Reject oversized email cleanly; no 500.
AI-FR03-11	forgot	SEC-05 SQL injection	A registered user exists	{"email":"' OR 1=1 --"}	400	[A1] Input is data, never SQL; no token for any user and exact error-only response.
AI-FR03-12	forgot	SEC-04 injection	None	{"email":"<script>alert(1)</script>@x.com"}	400	[A1] Reject and never reflect executable markup in response.
AI-FR03-13	forgot	email case	Registered test@eshop.com	{"email":"TEST@ESHOP.COM"}	200	[A4] Same account is found; exact success schema and six-digit token.
AI-FR03-14	forgot□reset	token replacement	One valid OTP already exists	Call forgot again, then reset once with old OTP and once with new OTP	400, then 200	SEC-07: new issue invalidates old OTP; new OTP resets password with [A2] exact success schema.
AI-FR03-15	forgot	SEC-07 entropy	Registered account; isolated repeatable test	Request 100 OTPs	200 each	Every response has exact schema; each OTP is exactly six random digits and not an obvious fixed/sequential value.
AI-FR03-16	reset	valid/schema	Fresh OTP belongs to email	{"email":"test@eshop.com", "resetToken":"123456", "newPassword":"Aa1!aaaa"} using actual OTP	200	[A2] Exact {"message":"Password reset successfully"}; no token/password fields.
AI-FR03-17	reset	state/email binding	Fresh OTP belongs to user A; user B exists	User B email with user A OTP and valid password	400	Exact [A1] error-only object; neither password changes.
AI-FR03-18	reset	token invalid	Account has a different active OTP	Correct email, "000000", valid password	400	Exact [A1] error-only object; password unchanged.
AI-FR03-19	reset	email required	Fresh OTP exists	Omit email	400	[A1] Reject; OTP remains unused.
AI-FR03-20	reset	token required	Account exists	Omit resetToken	400	[A1] Reject; password unchanged.
AI-FR03-21	reset	password required	Fresh OTP exists	Omit newPassword	400	[A1] Reject; OTP remains usable for a corrected request.
AI-FR03-22	reset	email format	Fresh OTP exists	"email":"bad-email" with valid OTP/password	400	[A1] Reject malformed email; no state change.
AI-FR03-23	reset	email type	Fresh OTP exists	"email":null	400	[A1] Reject null without 500; OTP not consumed.
AI-FR03-24	reset	token empty	Fresh OTP exists	"resetToken":""	400	[A1] Reject empty token; password unchanged.
AI-FR03-25	reset	token lower boundary	Fresh OTP exists	"resetToken":"12345"	400	SEC-07: reject fewer than six digits.
AI-FR03-26	reset	token upper boundary	Fresh OTP exists	"resetToken":"1234567"	400	FR-03: reject more than six digits.
AI-FR03-27	reset	token type	Fresh OTP value is 123456	"resetToken":123456 as JSON number	400	[A1] Reject non-string type; no coercion and no password change.

ID	endpoint	category	precondition	request/input	expected status	expected schema/behavior
AI-FR03-28	reset	password length	Fresh OTP exists	"newPassword": "Aa1!aaa" (7 chars)	400	Reject below minimum 8; OTP remains usable.
AI-FR03-29	reset	password uppercase	Fresh OTP exists	"newPassword": "aa1!aaaa"	400	Reject password with no uppercase letter.
AI-FR03-30	reset	password lowercase	Fresh OTP exists	"newPassword": "AA1!AAAA"	400	Reject password with no lowercase letter.
AI-FR03-31	reset	password digit	Fresh OTP exists	"newPassword": "Aaa!aaaa"	400	Reject password with no digit.
AI-FR03-32	reset	password special	Fresh OTP exists	"newPassword": "Aaa1aaaa"	400	Reject password lacking one of @ \$! % * ? &.
AI-FR03-33	reset	password type	Fresh OTP exists	"newPassword": null	400	[A1] Reject non-string password without 500; OTP not consumed.
AI-FR03-34	forgot□reset	SEC-07 expiry	OTP is older than configured lifetime [A3]	Correct email, expired OTP, valid password	400	Expired OTP is rejected and password stays unchanged.
AI-FR03-35	forgot□reset	SEC-01/SEC-07 reuse	One reset has succeeded	Reuse same email/OTP with another valid password; inspect test DB	400	Used OTP stays invalid; second password is not applied; first new password is stored as a one-way hash, never plaintext.

FR-09 Helper AI Log

- Tool: Codex helper AI (Banach)
- Date/time (Asia/Ho_Chi_Minh): 2026-08-18 12:25:59 +07

Original prompt

> You are a helper AI being guided by a third-year software engineering student. Inspect eshop-sut/api_specification.md and the relevant implementation only to understand FR-09 (POST /api/apply-coupon, with coupon usage state when relevant). Do not edit any files. Produce exactly 35 AI-generated test-case drafts in a compact Markdown table with columns: ID, endpoint, category, precondition, request/input, expected status, expected schema/behavior. Cover domain partitions for code, total_amount, user_id, coupon type/value/minimum/expiry/usage state, boundary and state transitions, SEC-01–SEC-07 where relevant, and exact response schema. Base expected behavior on the API specification, clearly flag any assumption, and keep the output concise enough to paste into an AI audit log.

Final output

Assumptions used in the drafts:

- The API specification guarantees 200 success with numeric discount_amount and final_amount. It does not define failure status codes or the complete failure schema. 400/401/403/404 and {error:string} below are assumptions based on common REST behavior and the current implementation.
- FR-09 says a valid JWT is required. Therefore, the authenticated JWT user should be authoritative; client user_id must not bypass usage limits.
- Percentage calculation follows the requirement: $\text{floor}(\text{total_amount} \times \text{discount_value} / 100)$.
- Coupon fixtures outside the seeded data require test setup.
- SEC-01, SEC-03, SEC-06, and SEC-07 are not applicable to this non-password, non-admin, non-profile, non-OTP endpoint. SEC-02, SEC-04, and SEC-05 are relevant.

ID	endpoint	category	precondition	request/input	expected status	expected schema/behavior
FR09-AI-001	POST /api/apply-coupon	Happy path/percent	Valid JWT; active SAVE10, 10%, min 300000, future expiry, usage 0/1	{"code": "SAVE10", "total_amount": 500000, "user_id": 1}	200	JSON contains numeric discount_amount: 50000, final_amount: 450000.
FR09-AI-002	POST /api/apply-coupon	Happy path/fixed	Valid JWT; active BIGBUY, fixed 50000, min 500000, future expiry, usage 0/1	{"code": "BIGBUY", "total_amount": 600000, "user_id": 1}	200	discount_amount: 50000, final_amount: 550000.
FR09-AI-003	POST /api/apply-coupon	Code partition/missing	Valid JWT	{"total_amount": 500000, "user_id": 1}	400*	{error:string}; coupon is not applied.
FR09-AI-004	POST /api/apply-coupon	Code partition/null	Valid JWT	{"code": null, "total_amount": 500000, "user_id": 1}	400*	{error:string}; no database/state change.
FR09-AI-005	POST /api/apply-coupon	Code boundary/empty	Valid JWT	{"code": "", "total_amount": 500000, "user_id": 1}	400*	{error:string}.
FR09-AI-006	POST /api/apply-coupon	Code boundary/whitespace	Valid JWT	{"code": " ", "total_amount": 500000, "user_id": 1}	400*	{error:string}; whitespace-only code is invalid.
FR09-AI-007	POST /api/apply-coupon	Code partition/unknown	Valid JWT; code absent from DB	{"code": "NO_SUCH_CODE", "total_amount": 500000, "user_id": 1}	404*	{error:string}; no discount.
FR09-AI-008	POST /api/apply-coupon	Code/case sensitivity	Valid JWT; only uppercase SAVE10 exists	{"code": "save10", "total_amount": 500000, "user_id": 1}	404*	Exact-code lookup assumed; must not silently select another coupon.
FR09-AI-009	POST /api/apply-coupon	Coupon state/inactive	Valid JWT; coupon exists with is_active=0	Valid code, total above minimum	404*	{error:string}; inactive coupon rejected.
FR09-AI-010	POST /api/apply-coupon	SEC-05/SQL injection	Valid JWT	code=" ' OR 1=1 -- "	404*	Treated as literal value; no coupon/data leak; parameterized query used.
FR09-AI-011	POST /api/apply-coupon	SEC-04/injection output	Valid JWT	code="<script>alert(1)</script>"	404*	JSON error must not execute/return active HTML; input must not be inserted through innerHTML by clients.
FR09-AI-012	POST /api/apply-coupon	Total partition/missing	Valid JWT; valid coupon	{"code": "SAVE10", "user_id": 1}	400*	{error:string}; no NaN or null monetary fields.
FR09-AI-013	POST /api/apply-coupon	Total partition/null	Valid JWT	total_amount: null	400*	{error:string}; no discount.
FR09-AI-014	POST /api/apply-coupon	Total boundary/zero	Valid JWT; coupon minimum > 0	total_amount: 0	400*	Reject below minimum; {error:string}.
FR09-AI-015	POST /api/apply-coupon	Total partition/negative	Valid JWT	total_amount: -1	400*	Negative order total rejected.

ID	endpoint	category	precondition	request/input	expected status	expected schema/behavior
FR09-AI-016	POST /api/apply-coupon	Total type	Valid JWT	total_amount:"500000"	400*	Numeric string rejected rather than coerced; {error:string}.
FR09-AI-017	POST /api/apply-coupon	Total type/object	Valid JWT	total_amount:{"value":500000}	400*	Object rejected; no server crash or non-finite result.
FR09-AI-018	POST /api/apply-coupon	Minimum boundary/below	Valid JWT; min 300000	total_amount:299999	400*	Below-minimum order rejected.
FR09-AI-019	POST /api/apply-coupon	Minimum boundary/equal	Valid JWT; active SAVE10, usage available	total_amount:300000	200	Requirement says >=; discount 30000, final 270000.
FR09-AI-020	POST /api/apply-coupon	Minimum boundary/above	Valid JWT; active SAVE10	total_amount:300001	200	Discount 30000 after integer flooring; final 270001.
FR09-AI-021	POST /api/apply-coupon	Decimal money	Valid JWT; percent 10%, min satisfied	total_amount:300000.50	200*	Formula applied consistently; assumed discount 30000, final 270000.50. Currency precision is unspecified.
FR09-AI-022	POST /api/apply-coupon	Expiry state/expired	Valid JWT; active coupon expired one second ago	Valid code and qualifying total	400*	{error:string}; expired coupon rejected.
FR09-AI-023	POST /api/apply-coupon	Expiry boundary/exact	Valid JWT; expired_at equals current instant	Valid code and qualifying total	400*	Requirement says current time must be before expiry; equality is expired.
FR09-AI-024	POST /api/apply-coupon	Expiry state/future	Valid JWT; expiry one second in future	Valid code and qualifying total	200	Discount calculated normally.
FR09-AI-025	POST /api/apply-coupon	Percent value/zero	Valid JWT; active percent coupon value 0	Total above minimum	200*	discount_amount:0; final_amount unchanged. Validity of zero-value coupons is unspecified.
FR09-AI-026	POST /api/apply-coupon	Percent value/100	Valid JWT; active percent coupon value 100	total_amount:500000	200*	discount_amount:500000, final_amount:0.
FR09-AI-027	POST /api/apply-coupon	Percent value/out of range	Valid JWT; percent coupon value 101 exists	Qualifying total	400*	Reject invalid coupon configuration; must not return negative final amount. Range validation is unspecified.
FR09-AI-028	POST /api/apply-coupon	Fixed value/greater than total	Valid JWT; fixed discount 600000, total 500000	Qualifying request	400*	Assumption: reject unsafe configuration/result; final amount must not become negative. Spec formula otherwise implies -100000.
FR09-AI-029	POST /api/apply-coupon	Coupon type/unsupported	Valid JWT; active coupon has type:"bonus"	Qualifying request	400*	Unsupported type rejected; it must not be treated as fixed.
FR09-AI-030	POST /api/apply-coupon	SEC-02/no authentication	Coupon valid; body claims existing user	No Authorization header; user_id:1	401*	{error:string}; body user_id cannot replace JWT authentication.
FR09-AI-031	POST /api/apply-coupon	SEC-02/invalid JWT	Valid coupon	Authorization: Bearer invalid; valid body	401*	{error:string}; no coupon details or usage information leaked.
FR09-AI-032	POST /api/apply-coupon	User ID type	Valid JWT for user 1	user_id:"abc"	400*	Invalid supplied type rejected, or ignored in favor of JWT subject; it must never select another usage record.

ID	endpoint	category	precondition	request/input	expected status	expected schema/behavior
FR09-AI-033	POST /api/apply-coupon	IDOR/user spoofing	JWT belongs to user 1; user 1 at limit; user 2 unused	Body sends user_id:2	400/403*	Must use JWT user 1 and deny application; changing user_id cannot bypass limit.
FR09-AI-034	POST /api/apply-coupon	Usage state transition	Valid JWT; coupon max 2	Apply at usage 1, record successful checkout, then apply again at usage 2	200 then 400*	At max-1, apply succeeds; after usage reaches max, {error:string} and no discount. Merely previewing before checkout should not consume a use.
FR09-AI-035	POST /api/apply-coupon	Exact response schema	Any valid percent request	Apply known coupon to known total	200	JSON object must contain numeric discount_amount and final_amount; values obey formula and final_amount=total-discount. The API spec does not define whether extra keys are allowed, so an exact whole-object key set cannot be asserted without clarification.

* = assumed because the API specification does not explicitly define that status, validation rule, or complete error schema.

Implementation observations for human audit: the current handler has likely defects against these expectations—it does not authenticate /api/apply-coupon, trusts optional body user_id, checks total_amount > min_order_amount instead of >=, and computes percent discount as $\text{floor}(\text{total} \times (1 - \text{discount_value}))$ rather than $\text{floor}(\text{total} \times \text{discount_value} / 100)$.

FR-12 Helper AI Audit Log

- Tool: Codex helper AI (Raman)
- Date/time (Asia/Ho_Chi_Minh): 2026-08-18 12:26:09 +07

Original prompt from the primary agent

> You are a helper AI being guided by a third-year software engineering student. Inspect eshop-sut/api_specification.md and the relevant implementation only to understand FR-12 access control. Use GET /api/admin/users as the representative selected API, with related admin endpoints only when needed to test authorization consistency. Do not edit any files. Produce exactly 35 AI-generated test-case drafts in a compact Markdown table with columns: ID, endpoint, category, precondition, request/input, expected status, expected schema/behavior. Cover Authorization header/token partitions, anonymous/user/admin roles, role escalation and IDOR, SEC-01–SEC-07 where relevant, and exact response schema. Base expected behavior on the API specification, clearly flag any assumption, and keep the output concise enough to paste into an AI audit log.

Exact final output

Assumptions used because the API specification does not define response bodies in detail:

- U is the exact successful user object inferred from the selected route: {id:number, name:string|null, email:string|null, role:string|null, login_attempts:number, locked_until:string|null, shipping_address:string|null}. No extra keys are allowed.
- Missing/malformed credentials return {"error":"Unauthorized"}; invalid JWT or insufficient role returns {"error":"Forbidden"}.
- Bearer is case-insensitive under normal HTTP authentication rules. A deleted/nonexistent resource returns 404 where the specification is silent.
- SEC-07's OTP entropy/lifetime rules are outside FR-12; only reset-token disclosure through the user-list response is relevant here.

ID	endpoint	category	precondition	request/input	expected status	expected schema/behavior
FR12-AI-01	GET /api/admin/users	Positive / FR-12	At least one user; valid admin JWT	Authorization: Bearer <admin>	200	JSON array of U; every stored user appears once.
FR12-AI-02	GET /api/admin/users	Empty boundary	No users; valid admin JWT	Normal request	200	Exact body [].
FR12-AI-03	GET /api/admin/users	SEC-01, SEC-07 / disclosure	Valid admin JWT	Normal request	200	Each item has exactly U; must omit password, reset_token, phone, and other secret/internal fields.
FR12-AI-04	GET /api/admin/users	SEC-04 / stored XSS	A user name/address contains <script>alert(1)</script>; admin JWT	Normal request	200	application/json; value is returned only as JSON text and never rendered/executed by the API response.
FR12-AI-05	GET /api/admin/users	Data boundary	User contains Vietnamese/Unicode and null optional fields	Normal admin request	200	Valid JSON U; Unicode preserved; locked_until/shipping_address remain JSON null where unset.
FR12-AI-06	GET /api/admin/users	SEC-02 / anonymous	None	No Authorization header	401	Exact {"error":"Unauthorized"}; no user data.
FR12-AI-07	GET /api/admin/users	SEC-02 / empty header	None	Authorization: ""	401	Exact unauthorized error; no user data.
FR12-AI-08	GET /api/admin/users	SEC-02 / missing credential	None	Authorization: Bearer	401	Exact unauthorized error; no user data.
FR12-AI-09	GET /api/admin/users	SEC-02 / missing scheme	Valid admin token exists	Authorization: <admin-token>	401	Exact unauthorized error; raw token is not accepted.
FR12-AI-10	GET /api/admin/users	SEC-02 / wrong scheme	Valid admin token exists	Authorization: Basic <admin-token>	401	Exact unauthorized error; JWT must not be accepted under Basic.
FR12-AI-11	GET /api/admin/users	Header syntax	Valid admin token exists	Authorization: Bearer <admin-token>	401	Exact unauthorized error under the specified single-space syntax.
FR12-AI-12	GET /api/admin/users	Header compatibility	Valid admin JWT	Authorization: bearer <admin-token>	200	JSON array of U; auth scheme comparison is case-insensitive.
FR12-AI-13	GET /api/admin/users	SEC-02 / malformed JWT	None	Bearer abc.def	403	Exact {"error":"Forbidden"}; no data.

ID	endpoint	category	precondition	request/input	expected status	expected schema/behavior
FR12-AI-14	GET /api/admin/users	SEC-02 / bad signature	JWT signed with another secret	Bearer <wrong-signature-token>	403	Exact forbidden error; no data.
FR12-AI-15	GET /api/admin/users	SEC-02 / tampering	Start with valid user JWT, edit payload without resigning	Change payload role to admin	403	Exact forbidden error; signature validation blocks tampering.
FR12-AI-16	GET /api/admin/users	SEC-02 / expiry	Expired admin JWT	Bearer <expired-admin-token>	403	Exact forbidden error; no data.
FR12-AI-17	GET /api/admin/users	SEC-02 / algorithm attack	Crafted unsigned JWT	JWT with alg:"none" and role:"admin"	403	Exact forbidden error; unsigned token rejected.
FR12-AI-18	GET /api/admin/users	SEC-03 / user role	Valid JWT with role:"user"	Bearer <user-token>	403	Exact forbidden error; no user list.
FR12-AI-19	GET /api/admin/users	SEC-03 / admin role	Valid JWT with role:"admin"	Bearer <admin-token>	200	JSON array of exact U.
FR12-AI-20	GET /api/admin/users	SEC-03 / missing claim	Validly signed JWT has id but no role	Bearer <no-role-token>	403	Exact forbidden error.
FR12-AI-21	GET /api/admin/users	SEC-03 / case boundary	Valid JWT with role:"ADMIN"	Bearer <uppercase-role-token>	403	Exact forbidden error; only exact role admin is authorized.
FR12-AI-22	GET /api/admin/users	SEC-03 / null claim	Valid JWT with role:null	Bearer <null-role-token>	403	Exact forbidden error.
FR12-AI-23	PUT /api/users/me then GET /api/admin/users	SEC-06 / role escalation	Logged-in normal user	PUT body includes {"name":"U", "shipping_address":"A", "phone":"1", "role":"admin"}, then obtain fresh JWT and GET	403 on GET	Profile may update allowed fields, but stored/token role remains user; admin list is not disclosed.
FR12-AI-24	POST /api/register then GET /api/admin/users	Role injection	New email	Register with extra "role":"admin", log in, then GET with issued JWT	403 on GET	Extra role is ignored; account/token role is user; exact forbidden error.
FR12-AI-25	DELETE /api/admin/users/:id	IDOR / own record	Normal user JWT; own ID exists	DELETE own ID	403	Exact forbidden error; own account remains unchanged.
FR12-AI-26	DELETE /api/admin/users/:id	IDOR / other record	Normal user JWT; another user exists	DELETE another user's ID	403	Exact forbidden error; target remains unchanged.
FR12-AI-27	DELETE /api/admin/users/:id	Positive authorization	Admin JWT; disposable target exists	DELETE target ID	200	Exact {"message":"User deleted"}; only target is removed.
FR12-AI-28	DELETE /api/admin/users/:id	SEC-05 / injection	Admin JWT; multiple users exist	ID path value 1%200R%201=1	404	No user is deleted; ID is handled as one parameter, never concatenated SQL.
FR12-AI-29	DELETE /api/admin/users/:id	SEC-02 / anonymous mutation	Target exists	No Authorization header	401	Exact unauthorized error; target remains unchanged.
FR12-AI-30	GET /api/admin/orders	SEC-03 / consistency	Valid normal-user JWT	Normal request	403	Exact forbidden error; no order data.
FR12-AI-31	PUT /api/admin/orders/:id/status	SEC-03 / mutation consistency	Normal-user JWT; pending order exists	{"status":"confirmed"}	403	Exact forbidden error; order status remains pending.
FR12-AI-32	POST /api/admin/coupons	SEC-03 / mutation consistency	Normal-user JWT	Valid coupon JSON	403	Exact forbidden error; coupon is not created.
FR12-AI-33	DELETE /api/admin/coupons/:id	SEC-03 / IDOR consistency	Normal-user JWT; coupon exists	DELETE coupon ID	403	Exact forbidden error; coupon remains unchanged.

ID	endpoint	category	precondition	request/input	expected status	expected schema/behavior
FR12-AI-34	POST /api/admin/import-products	SEC-03 / bulk mutation	Normal-user JWT	{"products":[{"name":"P","price":1000,"category_id":1}]}	403	Exact forbidden error; zero products inserted.
FR12-AI-35	POST /api/products	FR-12 / protected data API	Normal-user JWT	Valid product JSON	403	Exact forbidden error; product is not created because data-changing product APIs require admin role.